

PORTFOLIO · 01

Project Blackout

UE5.7 데디케이티드 서버 기반 멀티플레이 TPS

최승현 NET (네트워크·백엔드) 담당

- 게임플레이/렌더 폴리싱
- 분석 툴
- 매치메이킹 시스템

4인 (개발자 4명) · 8주 (2026.04 - 06) · UE5.7 / Nest.js / React

▶ 데모 영상

</> **GitHub** audidas/ProjectBlackout-code

타락한 약탈자



목차

01 프로젝트 개요

02 시스템 아키텍처

03 카메라 충돌

04 캐릭터 선택 3D 프리뷰

05 비행 보스 선회 AI

06 매치 상태 복제

07 매치메이킹 · 서버 할당

08 실시간 트러블슈팅

09 레벨 디자인 분석 툴

10 플레이 데이터 웹 뷰어

11 기술 스택 & 회고

프로젝트 개요

4인 협동 보스러쉬 TPS를 데디케이티드 서버 환경에서 구현한 팀 프로젝트
게임플레이·렌더 디테일을 맡아, 매치메이킹·분석 도구 시스템 담당

01 게임플레이 / 렌더 (클라)

TPS 카메라 충돌 · 캐릭터 프리뷰 · 진영 외곽선 · 사망 연출

02 네트워크 / 서버

매치 상태 복제 · 매치메이킹 · 실시간 트러블슈팅

03 분석 툴 (엔진 + 웹)

인에디터 레벨 분석 툴 · 웹 뷰어로 플레이 데이터 시각화



보스스테이지

시스템 아키텍처

UE 데디케이티드 서버 · 매칭 서버 · 웹 뷰어가 하나의 파이프라인으로 동작

UE5.7 클라이언트 / 데디케이티드 서버

C++ · GAS · Subsystem

게임플레이 · 렌더 · 매치메이킹 클라
플레이 데이터 샘플러 · 히트맵 툴



Nest.js 매치메이킹 서버

TypeScript · WebSocket · Redis

자동 매칭 · 데디케이티드 서버 풀 관리
세션 · heartbeat TTL



React 플레이 데이터 뷰어

deck.gl · Vite

히트맵 · 동선 · 재생
GPU 집계 시각화

데이터 흐름

매칭 → 데디케이티드 서버 할당 → 플레이 → 플레이 데이터 수집 → 뷰어 시각화

클라이언트 — 카메라 충돌 구현

[</> BlackoutPlayerCharacter.cpp:162-193](#)

문제 기본 카메라 세팅은 측면 벽에서도 카메라를 당겨 캐릭터가 화면을 가림

해결 직접 구면 sweep으로, 막힌 벽이 카메라 뒤쪽일 때만 당기고 측면 막힘은 무시

BEFORE / AFTER



측면 벽에 캐릭터가 화면 가림



뒤쪽 벽일 때만 당김: 측면에서 시야 유지

CODE

```
// 막힘 지점을 카메라 뒤축에 투영 → 뒤쪽 막힘만 당김
float Fwd = Dot(Hit.Loc - Pivot, -CamDir);
float Total = Dist(Hit.Loc, Pivot);
if (Fwd > Total * 0.5f)
    TargetLen = Max(Fwd - Buffer, MinArmLength);
```

함께 구현한
클라 기능

진영 외곽선
Custom Depth, 본인 조종 폰 제외

적 사망 연출 라이프사이클
폴 재사용 DMI 복원 + 서버 동기화

로컬 카메라 세이크
본인 조종 폰만 세이크

캐릭터 선택 — 클라별 3D 프리뷰

문제 단일 공유 RT + SceneCapture는 멀티 클라 PIE에서 남의 캐릭터 선택창이 섞여 보임

해결 클라마다 자기 RT에만 캡처하고, 프리뷰 폰은 복제 없이 로컬 처리

[</> BlackoutCharacterPreviewManager.cpp:13-36](#)

단일 공유 RT

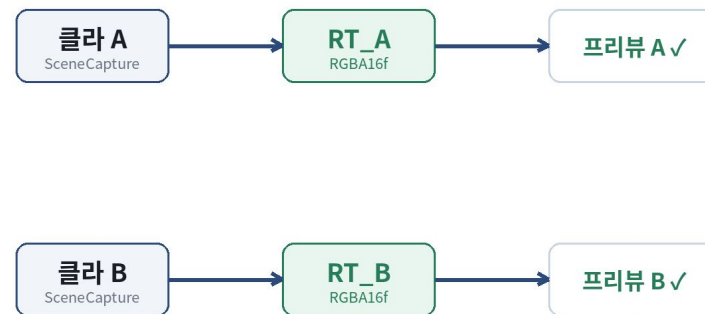
두 캡처가 같은 RT를 덮어씀



멀티 클라 PIE(한 프로세스)에서 race condition

per-client 동적 RT

클라마다 자기 RT에만 캡처



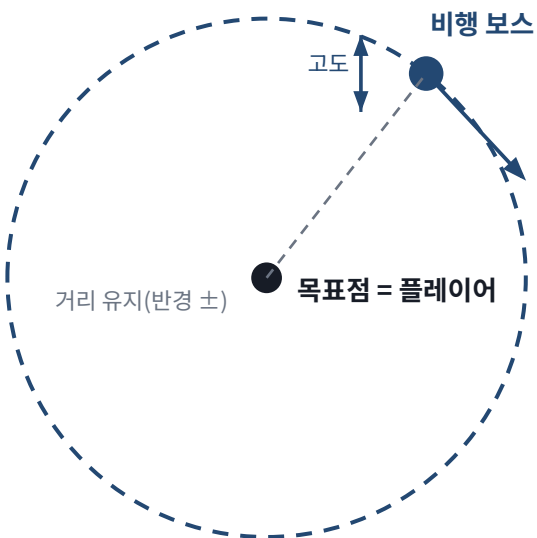
PRM_UseShowOnlyList로 프리뷰만 격리 · 공유 제거

비행 보스 선회 AI

[BSTask_FlyKite.cpp:62-78](#)

비행 보스가 플레이어와 거리를 유지하며 선회

선회 · 고도 · 반경 변화를 목표점 하나에 합쳐 단일 입력으로 처리



문제 선회 · 고도 · 반경을 따로 더하면 입력 방향이 희석돼 움직임이 깨짐

해결 셋을 목표점 좌표 하나에 인코딩 → 단일 이동 입력

효과 가변 반경 선회 + 부유로 비행 보스 자연스러운 움직임 구현

목표점 = 플레이어 위치 + 각도(θ) + 반경(R) + 고도(h)

● **각도** $\theta \leftarrow \theta + \omega \cdot \Delta t$
→ 원 궤도 선회

● **반경** 기본값 $\pm \sin$ 진동
→ 다가왔다 멀어졌다

● **고도** 랜덤 밴드 + $\sin$
→ 높이 변동

매치 상태 복제 동기화

매치 상태를 복제 프로퍼티로 전 클라에 전파하고, 서버(SetMatchState)·클라(OnRep) 양쪽이 동일 델리게이트를 호출해 HUD·UI를 별도 분기 없이 갱신

서버 · 권위

SetMatchState() — 매치 상태 확정

↓ 복제 · DOREPLIFETIME 프로퍼티

전 클라 · 복제 수신

OnRep_CurrentMatchState()

↓ 동일 델리게이트 호출

OnMatchStateChanged.Broadcast()

서버 · 클라 공통 경로

↓ 분기 없이 동일 UI 갱신

HUD · UI — 분기 없이 갱신

[</> BlackoutGameState.cpp:15, 59-74](#)

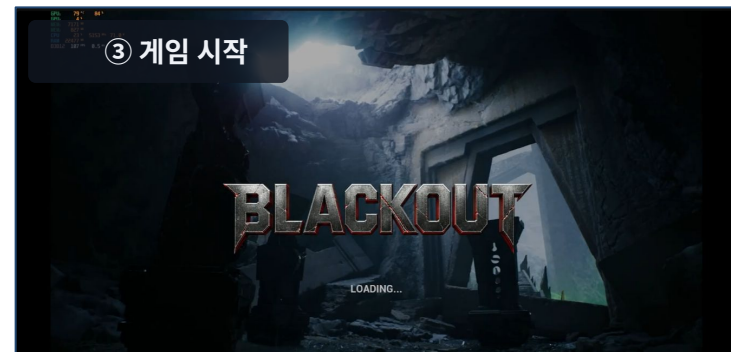
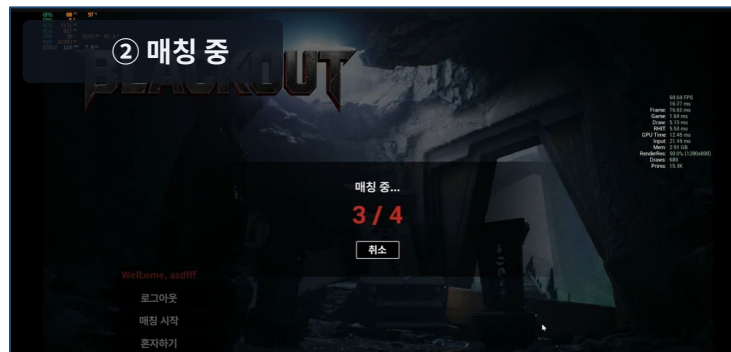
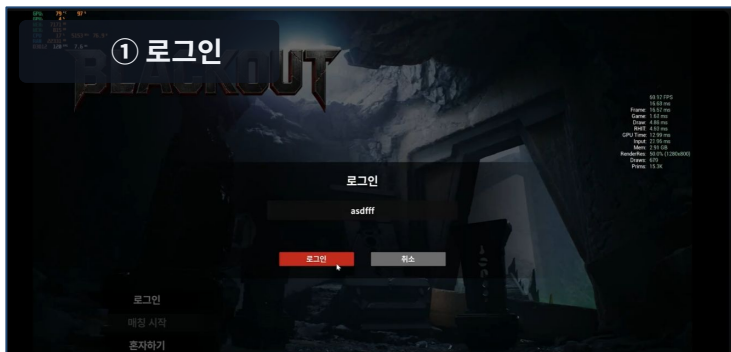
CODE

```
DOREPLIFETIME(ABlackoutGameState, CurrentMatchState);
OnMatchStateChanged.Broadcast(New); // 서버 경로
OnMatchStateChanged.Broadcast(CurrentMatchState); // 클라 OnRep
```



매치메이킹 — 자동 매칭 · 데디케이트드 서버 할당

매칭 버튼 → 자동 매칭(4인) → 데디 서버 할당 → 클라 자동 접속 · HTTP + WS 2채널, 상태 Redis



매칭 요청
클라 → API



자동 매칭
Redis · 4인



데디 서버 배정
유휴 서버



접속 정보
WS game_start



게임 진입
ClientTravel

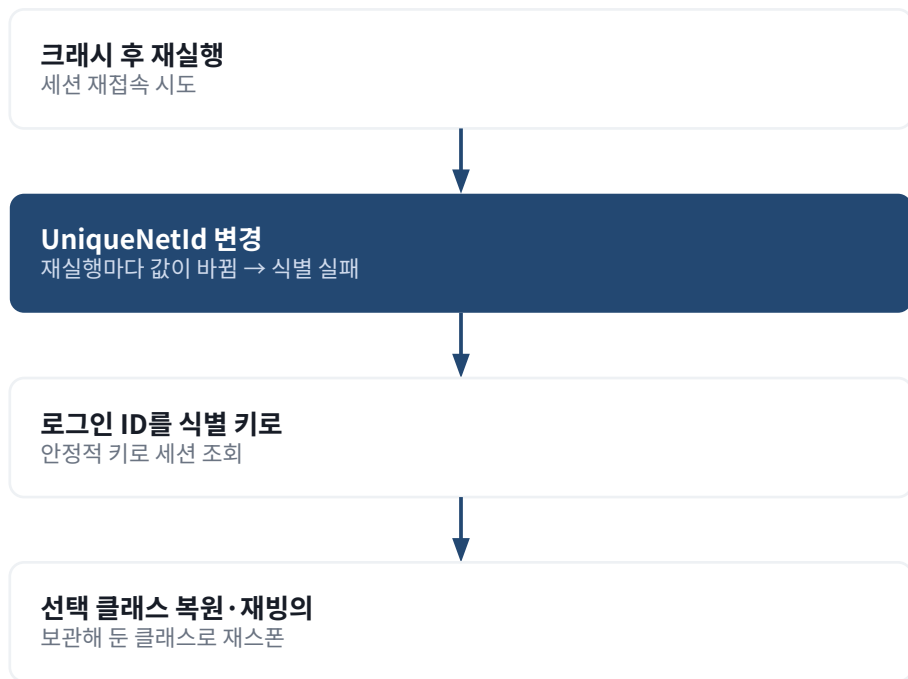
트러블슈팅 — 실시간 환경에서 마주친 문제

① 재접속이 끊기던 문제

문제 크래시 후 재실행하면 같은 세션 재접속 실패

원인 UniqueNetId는 재실행 시 값이 바뀜

해결 식별 키를 로그인 ID로 교체 → 연결 종료 시 선택 클래스를 보관, 재접속 시 복원·재빙의

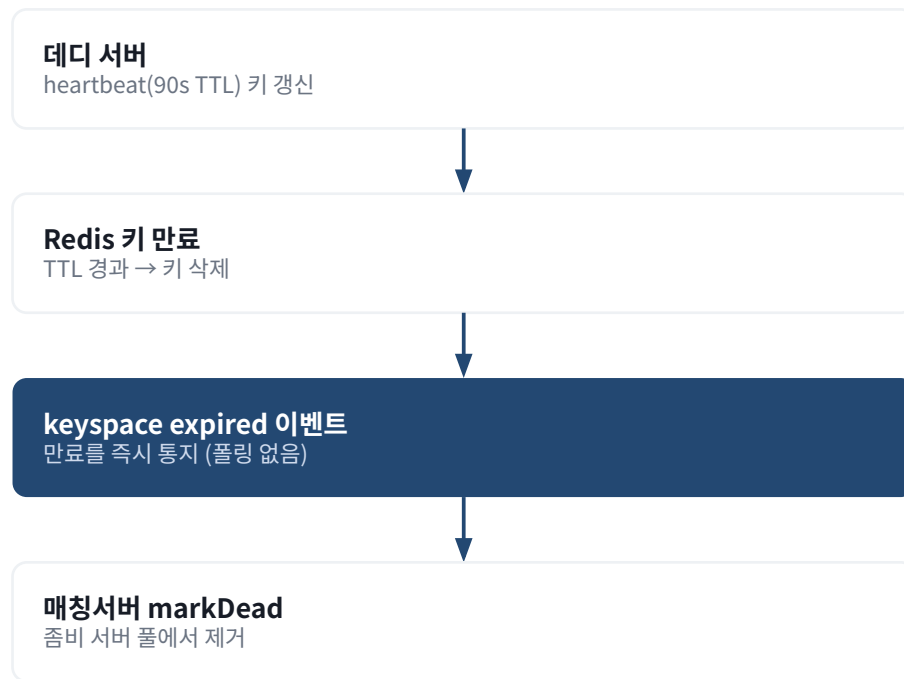


② 좀비 데디케이티드 서버를 폴링 없이 정리

문제 비정상 종료된 데디케이티드 서버가 사용 중 상태로 풀에 잔류

원인 데디케이티드 서버 생존 여부를 감지할 수단 부재

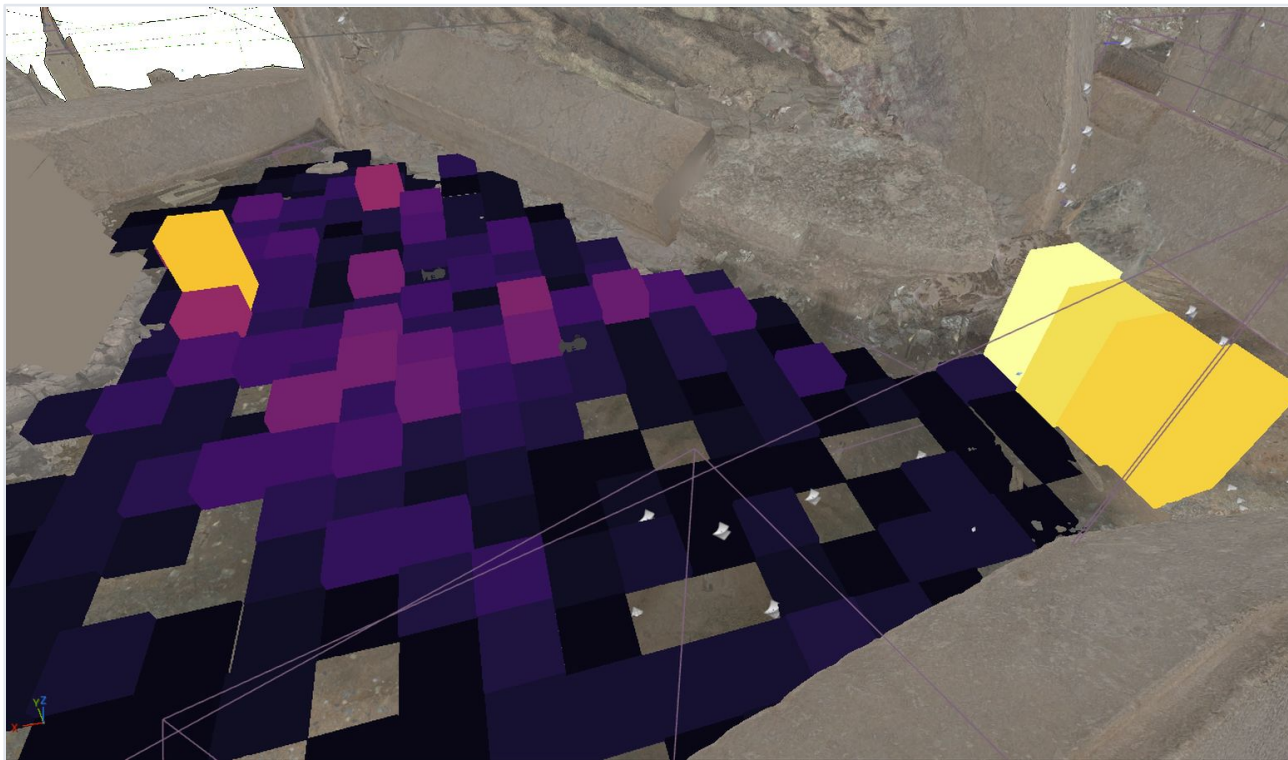
해결 heartbeat(90s TTL) 키 만료를 Redis Keyspace 이벤트로 감지해 즉시 정리 — 주기적 폴링 없음



레벨 디자인 분석 툴

가치 에디터에서 체류 분포를 3D 로 보고 레벨 수정을 판단 · 시각화

에디터 뷰포트



에디터 뷰포트 — ISM 막대로 빌드된 히트맵 (머문 시간이 길수록 밝음)

렌더

수천 셀을 개별 액터로 두면 에디터 정지 → ISM 단일 컴포넌트로 일괄 드로우

표현

막대 높이 = 체류시간, per-instance custom data로 색상표현

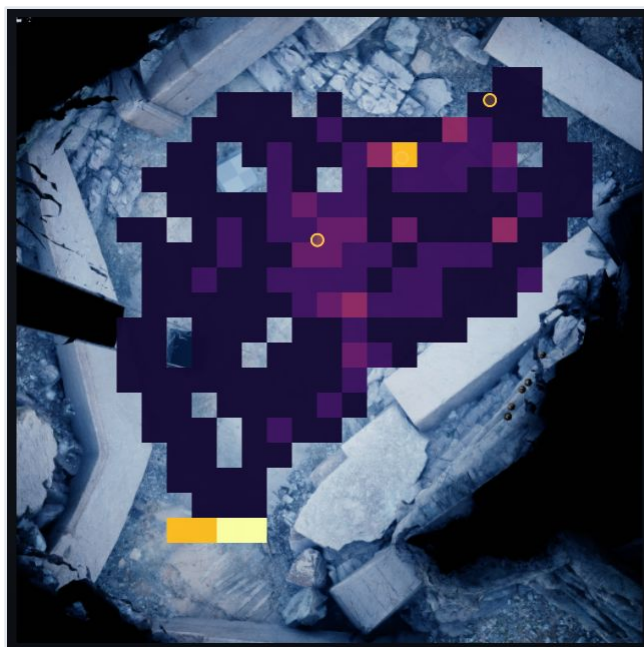
기준점

셀에 기록된 플레이어 Z 좌표 우선, 없으면 raycast 폴백

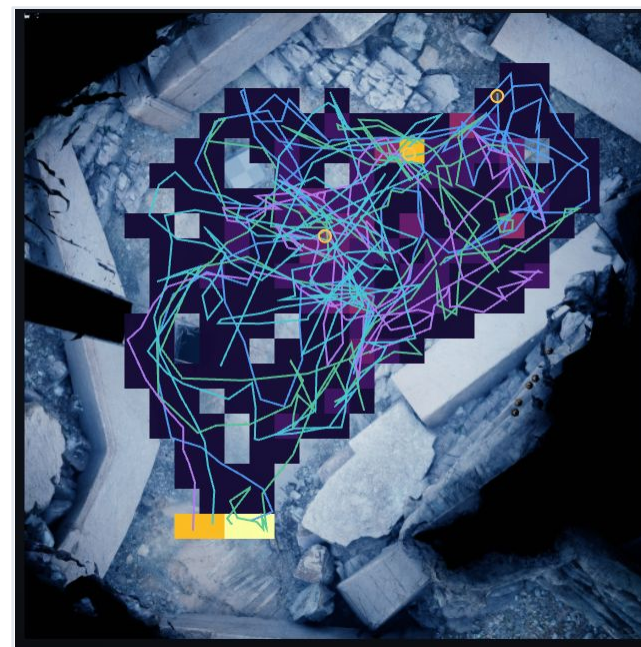
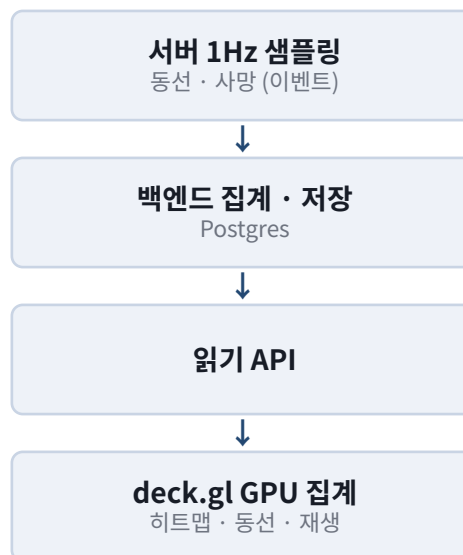
웹 뷰어 — 플레이 데이터 분석 파이프라인

분석 도구 서버 수집부터 GPU 집계 시각화까지, 풀스택 데이터 흐름

인에디터 툴 = 디자이너 개인용 · 웹 뷰어 = 팀 공용 대시보드 (수집은 에디터, 분석은 웹으로 분리)



■ 히트맵 — 체류 분포



■ 히트맵 + 동선

성과

갈 수 있지만 플레이어가 가지 않는 구역을 파악 → 맵 축소 판단 근거 제공

기술 스택과 배운 점

UE5.7 클라 / 데디케이티드 서버

C++17 · Unreal 5.7
GAS · Subsystem · ISM
Replication · ServerTravel

매치메이킹 백엔드

TypeScript · NestJS 11
ws · JWT
Redis (Keyspace Notif)
Postgres · Prisma · Docker

플레이 데이터 뷰어

TypeScript · React 19
deck.gl 9 · Vite
OrthographicView
집계 시각화

배운 점

- 실시간 통신 자체는 웹에서도 다뤘지만, 서버가 정한 상태를 복제로 모든 클라에 맞추는 게임 특유의 방식은 처음 경험했고 끊김·재접속 같은 문제를 직접 겪으며 익혔다
- 클라와 서버를 모두 다뤄봤기에, 매칭 서버가 필요한 순간 구조를 빠르게 떠올려 구현할 수 있었다
- 팀원들의 코드를 읽으며, 동작을 넘어 팀이 이어받기 쉬운 코드를 고민하게 됐다
- **향후 방향** 체류 데이터로 다음 판에 장애물을 자동 배치하는 기능까지 해보려 했으나 시간이 부족해 마무리하지 못했다. 이후 개인 프로젝트로 이어볼 계획이다

THANK YOU

감사합니다

최승현

▶ 데모 영상

</> **GitHub** audidas/ProjectBlackout-code